GENERATIVE AI ENGINEERING

Your First Agent Logic — The ReAct Framework

How agents think and act — iteratively, transparently, and powerfully.

Author: Cognizant

© 2025-2027 Cognizant | Private

cognizant

GENERATIVE AI ENGINEERING

Your First Agent Logic — The ReAct Framework

How agents *think* and *act* — iteratively, transparently, and powerfully.

ReAct = Reasoning + Action

A prompting technique that guides an LLM to think through a problem step by step — then act on that reasoning, loop back, and repeat until the goal is complete.

The Core Loop

Every ReAct agent operates on a single repeating pattern:

Thought → Action → Observation → Repeat

The cycle continues until the task is resolved — no guessing, no single-shot answers.

© 2025-2027 Cognizant | Private

Traditional LLM Interaction

Before agents, there was a single round-trip: you ask, the model answers. Understanding this baseline makes it clear exactly what agents add — and why they matter.

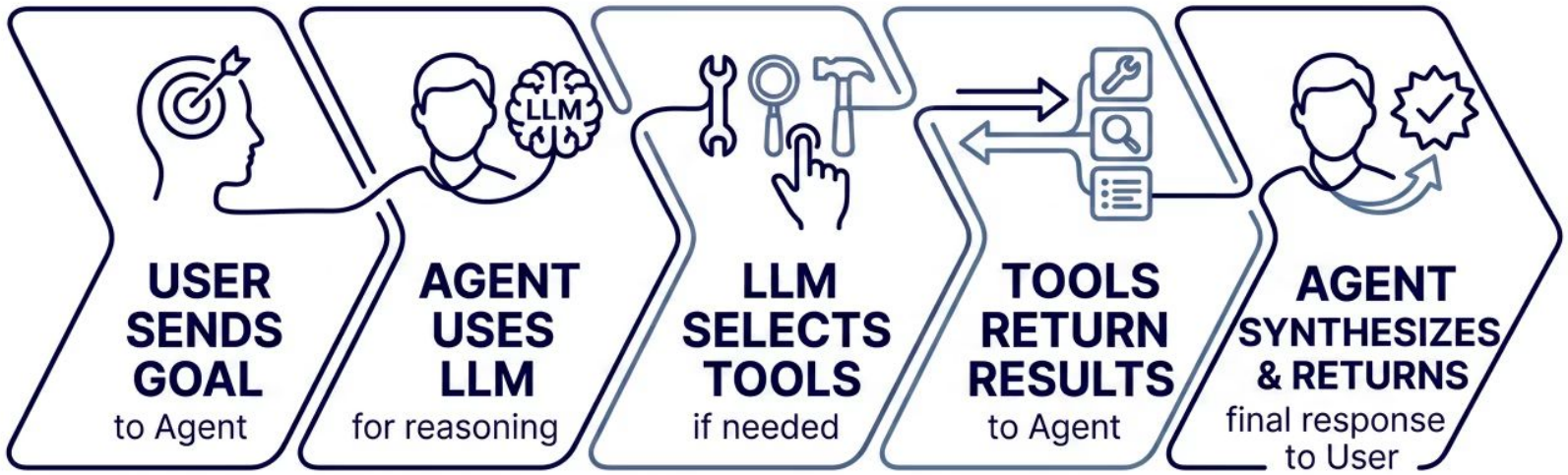


This model is fast and effective for simple, self-contained questions. But it has hard limits: **no iteration, no tools, no memory between steps**. When a task requires multiple steps, external data, or decision-making under uncertainty, a single pass simply isn't enough.

© 2025-2027 Cognizant | Private

Agent-Based Interaction

With an agentic system, the user no longer talks directly to the LLM. Instead, an **agent** sits in between — reasoning, delegating, and orchestrating until the task is done.



The agent coordinates everything: which tools to call, when to reason again, and when the response is ready. The user simply gets a final, fully-resolved answer.

© 2025-2027 Cognizant | Private

cognizant

Components of an Agentic System

Five distinct layers work together to make an agent function. Each has a clear role — and together they form a complete, self-directing system.

1	**User** Provides goals, questions, and tasks. The starting point of every agent interaction.
2	**Agent** The decision-maker and coordinator. Routes instructions, manages state, and drives the loop.
3	**LLM** The reasoning engine. Selects actions, interprets observations, and plans next steps.
4	**Tools** Extend capability beyond text — web search, database queries, APIs, calculators, file operations.
5	**Orchestration Framework** Manages communication, execution flow, shared state, and the iterative looping logic.

© 2025-2027 Cognizant | Private

The ReAct Execution Loop

ReAct isn't a single step — it's a **continuous reasoning cycle**. The agent thinks, acts, observes the result, and uses that new information to think again. This loop repeats until the goal is fully achieved.



Thought

Agent reasons about what's needed next and forms a plan.



Action

Agent invokes a tool or executes a specific task.



Observation

Agent receives the result and incorporates it into context.



Repeat

New thought begins — loop continues until goal is complete.

Pattern: Thought → Action → Observation → Thought → Action → Observation → ... until done.

© 2025-2027 Cognizant | Private

Seeing ReAct in Action

Here's what a single ReAct loop looks like in practice — a concrete trace of how the agent's internal monologue maps to external actions and incoming observations.

🧠 Thought #1

The user wants the current weather in Tokyo. I don't have real-time data, so I need to call a weather API tool.

⚡ Action #1

`weather_tool(location="Tokyo")` — Agent invokes the external weather tool.

🔍 Observation #1

Tool returns: *"Tokyo: 22°C, partly cloudy, humidity 65%."*

🧠 Thought #2

I now have the data I need. I can formulate a complete, accurate response for the user.

✅ Final Response

Agent returns the answer — loop terminates because the goal is achieved.

© 2025-2027 Cognizant | Private

Why ReAct Matters

ReAct closes the gap between an LLM's internal knowledge and the dynamic, real-world tasks users actually care about. It's not just a prompting trick — it's the architectural foundation of capable agents.



Planning + Execution, Unified

ReAct combines reasoning and action in a single workflow — no separate planning phase needed. The agent plans as it executes.



Tool Use on Demand

When internal knowledge falls short, the agent reaches for tools — APIs, search engines, databases — exactly when needed.



Transparent Reasoning

Intermediate thought steps are exposed, making agent behavior interpretable, debuggable, and auditable by developers.



Multi-Iteration Tasks

Complex goals that require multiple steps, tool calls, or course corrections are handled gracefully through the repeating loop.

© 2025-2027 Cognizant | Private

ReAct vs. Simple LLM: A Direct Comparison

The difference between a plain LLM call and a ReAct agent isn't just technical — it's a fundamentally different capability profile.

Dimension	Traditional LLM	ReAct Agent
Interaction model	Single-pass request/response	Iterative loop until goal complete
Tool access	None — text only	Full tool use on demand
Reasoning visibility	Hidden — output only	Exposed via Thought steps
Complex task handling	Limited to training knowledge	Dynamic, multi-step execution
Error correction	No — one shot	Yes — observes and adapts

© 2025-2027 Cognizant | Private

ReAct Teaches an Agent How to Think and Act

What ReAct Adds to a Simple LLM

01

Reasoning

Explicit thought steps before every action.

02

Tool Usage

Real-world capability beyond training data.

03

Iterative Loops

Repeat until the task is genuinely complete.

04

Orchestration

Coordinated state management across the loop.

The Universal Pattern

No matter how complex the task, every ReAct agent follows the same fundamental loop:

> **Thought → Action → Observation → Repeat until goal completion**

This pattern is the foundation for everything you'll build in this course. Master it here, and every agentic framework you encounter next will feel familiar.

© 2025-2027 Cognizant | Private

What's Next

You now understand *how* a ReAct agent decides what to do at each step. Next, we'll go deeper — looking at how these loops are implemented in real frameworks, how tools are defined and registered, and how to build your own agent from scratch.

✅ MCP + Agent Architecture

Foundation: protocols and system design.

▶️ Tool Definition

How to build, register, and invoke tools.

⬜ Evaluation + Debugging

Test, trace, and improve agent behavior.

✅ ReAct Framework

Reasoning-and-acting loop — this module.

⬜ Agent Implementation

Build a full ReAct agent from scratch.

Keep this pattern close: **Thought → Action → Observation → Repeat**. Every agentic framework you'll encounter is a variation of this core loop.

© 2025-2027 Cognizant | Private